

SENG 474 Data Mining: Assignment 1 Report

Joseph Bresee V01005288

June 11, 2026

1 Introduction and Model Selection

This report provides all details of the data mining pipeline implemented for the wine quality dataset, which included data exploration, pre-processing, model selection and implementation, hyper-parameter experimentation, and model evaluation. Starting with the choice of models I chose to implement, I will describe the insight I gained in initial data exploration, pre-processing steps, model performance analysis and hyper-parameter tuning, a comparative analysis of the two classification models I chose, and ending in a summary of my findings.

- **Ridge Regression (Scikit-Learn):** Chosen for the regression task to predict quality scores. The regularization penalty resists overfitting by shrinking the coefficient values without setting them to zero, which reduces variance, handles collinearity better than linear regression, but introduces a small bias as a tradeoff. I chose this model because I wanted to see its performance in on chemical properties, which can often be collinear.
- **Logistic Regression (From-Scratch):** As a linear model, it is simple, computationally inexpensive, and has reliable performance on binary classification tasks.
- **Multi-Layer Perceptron (From-Scratch):** Chosen to find complex, non-linear relationships between the chemical features and the target class that a simple linear model might fail to see. Also chosen to learn how added complexity of a model influences performance of the model in relation to more simple models like regularized linear / logistic regression.
- **K-Means Clustering (Scikit-Learn):** Implemented as an unsupervised learning tool to identify clusters in the data prior to classification, helping outlier detection and structural analysis.

2 Data Exploration Insights

Initial exploration of the dataset (6,821 samples, 13 columns) yielded several insights that informed the decisions made later in the pre-processing phase of the pipeline. A feature correlation matrix revealed that *chlorides* and *total sulfur dioxide* were closely associated with the target class. Additionally, *volatile acidity* demonstrated near-zero correlation with all other features.

An exploratory K-Means clustering pass was used to measure the distance from each data point to its nearest centroid. By mapping these distances on a logarithmic histogram, a clear threshold emerged (around a distance of 8), indicating the presence of extreme synthetic outliers.

Additionally, a search for missing values indicated that the *alcohol* column was missing data for approximately 18.87% of the samples.

3 Data Pre-processing

To ensure model performance, my pre-processing pipeline included:

- **Imputation:** The missing values in the *alcohol* feature were filled using median imputation. Because the missing rate was under the 20% threshold, imputation was deemed safe, and the median was chosen over the mean to prevent skewing from extreme outliers.
- **Feature Elimination:** Domain knowledge and correlation heatmaps justified dropping two features. *Volatile acidity* was removed due to the presence of physically impossible negative values, high artificial noise, and a lack of predictive correlation which was deemed ultimately not useful for the downstream models. *Free sulfur dioxide* was dropped due to its high collinearity with *total sulfur dioxide*, which possessed a stronger correlation with the target class indicating that *Free sulfur dioxide* was redundant and unnecessary for training.
- **Data Transformation:** Logarithmic transformations ($\log_{10}$) were tested on right-skewed columns. The transformations improved the distributions of *sulphates* and *residual sugar* so the transformations were kept. Transformations on *chlorides* and *total sulfur dioxide* worsened their distributions and the transformation was not kept.
- **Outlier Removal:** An (IQR) filter ($Q1 - 3 \times IQR$ to $Q3 + 3 \times IQR$) was applied to remove extreme outliers. The noisy *class* feature was explicitly excluded from this to preserve its natural Gaussian noise since its a target feature.
- **Scaling:** Features were normalized using Min-Max scaling to map all domains to $[0, 1]$, ensuring stable gradient descents for the custom neural network and logistic regression models.

4 Performance Analysis and Hyper-parameter Tuning

Following a stratified 80/20 train-test split, hyper-parameter tuning was conducted using K-Fold Cross-Validation ($k = 5$) to analyze model generalization.

4.1 Ridge Regression

Experiments varied the regularization strength (α) and the training set size. The validation Mean Squared Error (MSE) remained stable for $\alpha \leq 1.0$ but increased sharply for $\alpha \geq 10$, indicating significant underfitting as the model was too heavily penalized. Evaluating across different fractions of the training set demonstrated that the model starts to generalize well with 70% of the data, eventually plateauing. The final model ($\alpha = 1.0$) achieved a Train MSE of 0.0163 and a Test MSE of 0.0159, indicating great generalization with no signs of overfitting.

4.2 Logistic Regression

The custom Logistic Regression model was tested across various learning rates. The error curve illustrated that learning rates between 0.5 and 1.0 minimized validation error well. The training fraction experiment mirrored the Ridge model, stabilizing at around 80% data utilization. Using a learning rate of 0.5 over 1000 iterations, the model achieved a Train Error of 0.0158 and a Test Error of 0.0128.

4.3 Multi-Layer Perceptron (MLP)

The MLP was tuned for learning rate and hidden layer architecture. Due to compute limits, expanding the network beyond (128, 64, 32) yielded diminishing returns relative to execution time. An architecture of a single 128-node hidden layer provided the best efficiency-to-accuracy tradeoff. A high learning rate of 1.0 proved optimal, minimizing validation error to 0.0066. At learning rates of 2.0 and above, the model began to overshoot and overfit. The finalized MLP resulted in a Train Error of 0.0066 and a Test Error of 0.0072.

4.4 K-Means Clustering

Testing K-Means across a range of k clusters produced a standard elbow plot, flattening precisely at $k = 3$. Varying the maximum iterations showed a flat inertia curve, meaning the centroids always converged before the default 300 iterations.

5 Comparative Analysis: Classification Models

Both classification models successfully generalized to the test set. However, the self-implemented Multi-Layer Perceptron significantly outperformed the Logistic Regression baseline. The Logistic Regression model achieved a test error of 1.28%, while the MLP achieved a test error of 0.72%, which is nearly half the error rate. Performance time-wise is also worth noting in this investigation across the two models; Logistic Regression was able to run much faster than the MLP, on the order of seconds, while the MLP ran on the order of minutes.

The performance gap with Logistic Regression’s strictly linear decision boundary was realized when compared against the complex MLP model that was able to classify with more nuance past linear decision making. The chemical composition data determining the wine class features non-linear feature relations. The MLP successfully mapped these non-linear sub-features, yielding a much more precise classification boundary ultimately yielding a much lower error.

6 Conclusion

The results of this data pre-processing and model hyper-parameter tuning experimentation demonstrates the importance in both proper pre-processing and hyper-parameter tuning. By imputing missing cells, filtering synthetic noise, logically dropping redundant features, and normalizing distributions, the models were able to receive high-quality data, in which the models were able to train on and result in low errors across all model types. While the Multi-Layer Perceptron proved to be the superior predictive model due to its non-linear mapping capabilities,

the baseline models still achieved low error rates, which proves the validity of the dataset pre-processing.

Additionally, training these models in practice gave real and practical insight to data, hyper-parameter and model choice, based on how much data you have and need, how efficient your model needs to be (i.e. if you have time or machine constraints), and how experimentation on models before a final run on the dataset was able to lower errors further by knowing the best hyper-parameter values to use in the final run.